Appendix of "QTem: Bounded-Error Compressed Data Movement for Temporal Graph Neural Network Training"

Weiwei Ye
The University of Tokyo
Japan
wwye@g.ecc.u-tokyo.ac.jp

Yankai Chen
MBZUAI
United Arab Emirates
yankaichan3@gmail.com

Zhen Wang
Huzhou Normal University
China
zhenwangrs@gmail.com

Dongyuan Li
The University of Tokyo
Japan
lidy@csis.u-tokyo.ac.jp

This appendix accompanies the paper. Sections, tables, figures, equations, and statements that are not part of this document refer to the paper; tables and figures continue its numbering.

A PROOFS

We use the notation of Section 4, where the proofs of Theorem 4.1, Corollary 4.2, and Proposition 4.4 are given.

Proof of Proposition 4.3. Let $\hat{\boldsymbol{\alpha}}$ denote the attention weights computed from the decoded inputs, that is, the softmax of $\hat{\mathbf{z}}$. Then

$$\mathbf{h} - \hat{\mathbf{h}} = \textstyle\sum_n (\alpha_n - \hat{\alpha}_n)\, \mathbf{W}_V \mathbf{x}_n + \sum_n \hat{\alpha}_n\, \mathbf{W}_V (\mathbf{x}_n - \hat{\mathbf{x}}_n).$$

The second sum is a convex combination, so its norm is at most $\|\mathbf{W}_V\|_2\, \varepsilon$. For the first,

$$\Big\| \sum_n (\alpha_n - \hat{\alpha}_n) \mathbf{W}_V \mathbf{x}_n \Big\|_2 \le \|\boldsymbol{\alpha} - \hat{\boldsymbol{\alpha}}\|_1 \max_n \|\mathbf{W}_V \mathbf{x}_n\|_2.$$

The Jacobian of the softmax at any point is $\mathrm{diag}(\boldsymbol{\sigma}) - \boldsymbol{\sigma}\boldsymbol{\sigma}^\top$, and for every $\mathbf{v}$ it satisfies $\sum_i \sigma_i |v_i - \boldsymbol{\sigma}^\top \mathbf{v}| \le 2\|\mathbf{v}\|_\infty$; integrating along the segment from $\mathbf{z}$ to $\hat{\mathbf{z}}$ yields $\|\boldsymbol{\alpha} - \hat{\boldsymbol{\alpha}}\|_1 \le 2\|\mathbf{z} - \hat{\mathbf{z}}\|_\infty$. Finally, $|z_n - \hat{z}_n| \le \|\mathbf{q}\|_2 \|\mathbf{W}_K\|_2\, \varepsilon / \sqrt{d'}$ by the Cauchy–Schwarz inequality. Combining the three bounds proves the claim. If the query is itself computed from a decoded root feature, the same argument adds a term proportional to its input error, again independent of k. □

B CONFIGURATIONS

All models use the single-layer architectures of the benchmark [1]: the 10 most recent temporal neighbors, two attention heads, 100-dimensional hidden states and time encodings, GRU memory for TGN, and a ten-entry mailbox with a transformer memory updater for APAN. The checkpoint with the best validation AP from the fourth epoch on is evaluated on the test split. In Table 2, the epoch times and the AP of TGL, SIMPLE, and ETC, as well as the traffic in Table 14, use the default configurations of the benchmark: five epochs, a learning rate of 10^{-4}, batches of 5,000 events for TGAT and of 1,000, 1,500, 2,000, and 2,500 events for TGN and APAN on LastFM, Wiki-Talk, Stack-Overflow, and GDELT, a dropout of 0.2 for TGN and 0.1 otherwise, and, on Wiki-Talk, 172-dimensional random edge features [1]. Table 4 lists the settings of the paired runs of Section 6.2, which also give the AP of QTem in Table 2; we fixed them while developing our pipeline, partly on development runs whose test AP we inspected, and they differ from the default configurations in 8 of the 12 configurations (on Wiki-Talk, these runs draw no random edge features). Every table is encoded by the rule of Section 3.6.

Table 4: Settings of the paired runs and of the per-seed runs in Table 6.

Graph	Model	Batch	LR	Dropout	Epochs	Codec
LastFM	TGN	1000	$2 \cdot 10^{-4}$	0.2	5	8-bit, $b = d$
	TGAT	5000	$2 \cdot 10^{-4}$	0.1	5	8-bit, $b = d$
	APAN	1000	10^{-4}	0.3	5	8-bit, $b = d$
Wiki-Talk	TGN	1500	10^{-4}	0.2	5	8-bit, $b = 4$
	TGAT	1500	10^{-4}	0.1	5	8-bit, $b = 4$
	APAN	1500	10^{-4}	0.1	5	8-bit, $b = 4$
Stack-Overflow	TGN	2000	10^{-4}	0.2	5	8-bit, $b = 4$
	TGAT	2000	10^{-4}	0.1	5	8-bit, $b = 4$
	APAN	2000	10^{-4}	0.1	5	8-bit, $b = 4$
GDELT	TGN	2500	10^{-4}	0.2	5	bit planes
	TGAT	5000	10^{-4}	0.1	10	bit planes
	APAN	2500	10^{-4}	0.1	5	bit planes

C BASELINES

TGL, SIMPLE, and ETC. Their AP in Table 2 comes from complete training runs on our server with the data files, chronological split, and default configurations of the benchmark (Appendix B), one negative destination per positive edge drawn uniformly from all nodes, AP averaged over batches, and the seeds 0, 1, and 2. Every run trains five epochs and tests the checkpoint with the better validation AP of the last two epochs; memory-based models replay the training and validation events before testing. All three run their released code: TGL at commit 241b336 with a reader for the data files of the benchmark, ETC at commit db04ac6, and SIMPLE at commit 3c29902 with its default buffer of 10% of the data. GDELT's binary attributes are kept as bytes and converted to 32-bit floats after every gather, which leaves every value a model reads unchanged.

Orca and Zebra. Their AP in Table 2 is the one reported for TGN, the model they implement [1]. Their epoch times were measured on our server with their released training loops (Orca at commit 2f6d5b4, Zebra at 8e72d65), data loaders that read the same files, and their default TGN models with two graph attention layers over ten neighbors and 100-dimensional memories and embeddings.

D ADDITIONAL ACCURACY RESULTS

Table 5 gives the standard deviations over seeds of the AP in Table 2 and, in its last column, the AP of the full-precision pipeline trained with the same settings and seeds as QTem. Table 6 lists the paired runs on the benchmark graphs and Table 7 those on the two real-feature graphs behind Figure 4. Both arms of a pair share the code base, the initialization, the sampler, and the negatives; only the representation of the static features differs. 8-bit codes do not make

training less stable: the median seed-to-seed standard deviation of QTem over the cells is 0.20 points, against 0.24 at full precision. Wikipedia and Reddit use TGL's settings for these graphs [3] (batch size 600, learning rate 10^{-4}, ten epochs, dropout 0.2 for TGN and 0.1 otherwise) with the same single-layer models and checkpoint rule. The control arm trains the same models with every static feature replaced by zero; it confirms that the models exploit the information in the LIWC features, most clearly TGN and TGAT on Reddit, and QTem preserves this information.

Stack-Overflow-APAN. This configuration is the least stable one: at full precision, its test AP ranges from 67.9 to 78.7 across seeds, and it yields the largest paired difference (mean −7.16 points, $p = 0.25$). Two controls show that this difference stems from where training ends, not from the 8-bit inputs. A repeated full-precision run of seed 0 with identical code, configuration, seed, and inputs ends 3.9 points from the original, because nondeterministic memory updates make two runs diverge from the first epoch on. Evaluating the trained models of both pipelines for seeds 0 and 1 with the inputs of the other pipeline moves their test AP by at most 0.44 points.

Ranking with 49 negatives. We re-evaluated all 90 paired checkpoints of Tables 6 and 7 on the test split with 49 random negatives per positive, each with the representation it was trained with, and computed the mean reciprocal rank of every positive among its 50 candidates (Table 8). After Holm's correction for the 15 cells, no cell differs significantly (smallest adjusted $p = 0.12$), and over all 45 pairs, the MRR of QTem differs from full precision by +0.4 points on average (95% CI [−1.5, 2.3]).

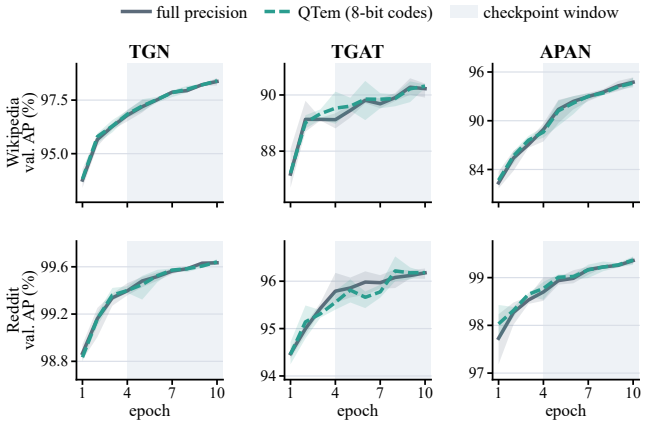


Figure 9: Training dynamics on real features: validation AP after every epoch for the full-precision pipeline and QTem (mean over three seeds; bands: min–max). The shaded region is the window from which the checkpoint is selected.

Training dynamics. Figure 9 follows the validation AP of the real-feature runs through all ten epochs. In every panel, the largest gap between the mean curves of the two pipelines, at most 0.54 points, is smaller than the spread across seeds of the full-precision pipeline at the same epoch.

E LOCK-STEP REPLAY DETAILS

Table 9 summarizes the LastFM replay of Section 6.3 per model and codec, averaged over the three trained models. The input error is the relative error of the event features that enter messages and mailboxes; the logit deviation and the share of flipped link predictions compare every batch with the deterministic full-precision copy; and the memory deviation is the relative Frobenius distance of the whole memory table, sampled every two batches. The last row of each block compares two copies of the unmodified full-precision pipeline. The ratio of memory deviation to input error is 0.14 for TGN and 0.068 for APAN, nearly identical for all three codecs: the memory error is a linear response to the input error with a model-specific gain below one, which the mean-square gains of the trained updaters predict (Appendix F).

Real features. Figure 10 and Table 10 list the replay of Wikipedia and Reddit behind Section 6.3. Each row averages the three models trained at full precision with different seeds; deviations are measured against the deterministic full-precision copy over the test segment, and ΔAP is the change of the test AP of the replayed model. With 8-bit codes, no replayed model changes its test AP by more than 0.006 points, and fewer than 0.12% of link predictions change sign, whereas destroying the features changes predictions substantially, above all on Reddit-TGAT, whose test AP drops by 23 points with zeros and by 12 with shuffled rows. Four-bit codes move the logits of the Reddit models by up to 89%, which is why QTem uses 8-bit codes.

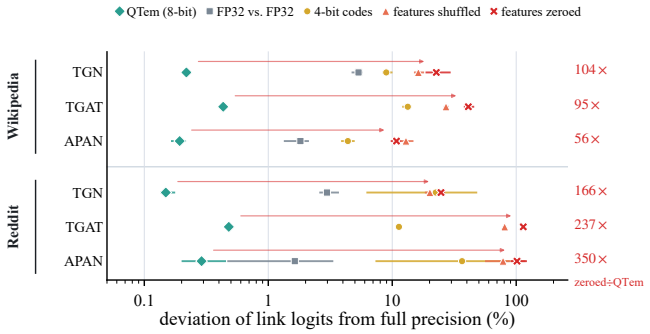


Figure 10: Feature sensitivity of trained models on real features: relative deviation of the link logits from the full-precision copy over the test segment of a lock-step replay (mean over three trained models; bars: min–max) when the models read QTem codes, 4-bit codes, shuffled feature rows, or zeros instead of the original features, and between two runs of the unmodified memory-based models. Right: ratio between zeroed features and QTem.

F CONTRACTION OF TRAINED MEMORY UPDATERS

Section 4 examines the premise of Proposition 4.4 on trained models. We replay the full LastFM stream through the TGN and APAN models trained with seed 0 (Table 6), with duplicate memory writes resolved deterministically as in Section 6.3. On every 25th batch, for up to 500 (TGN) or 100 (APAN) positive root nodes, we compute

Table 5: Test AP (%) in Table 2: mean ± standard deviation over the seeds 0, 1, and 2. †Settings of Table 4 (GDELT: seeds 0, 1, and 7): QTem and the full-precision pipeline (fp32) of the same runs.

Graph	Model	TGL	SIMPLE	ETC	QTem†	fp32†
LastFM	TGN	80.44±1.88	81.68±0.26	80.61±0.16	84.07±0.57	83.86±0.98
	TGAT	67.56±0.63	67.60±0.67	67.37±0.56	69.61±0.48	69.52±0.50
	APAN	69.95±0.47	68.72±3.31	69.46±0.15	69.20±0.19	69.45±0.24
Wiki-Talk	TGN	92.97±0.12	84.14±0.10	93.16±0.39	89.94±0.89	90.03±3.28
	TGAT	86.16±0.09	82.15±0.13	86.06±0.06	86.87±0.07	86.85±0.08
	APAN	82.73±3.22	76.19±0.53	84.99±2.63	86.43±0.63	86.62±0.38
Stack-Overflow	TGN	84.58±1.69	86.93±0.50	86.45±0.92	85.83±0.91	85.93±0.04
	TGAT	87.71±0.06	87.70±0.27	87.74±0.17	87.66±0.05	87.44±0.08
	APAN	67.96±2.01	72.80±0.07	69.15±0.90	64.78±2.05	71.94±5.85
GDELT	TGN	97.41±1.04	96.49±2.82	97.87±0.24	97.23±0.89	97.84±0.46
	TGAT	98.07±0.01	98.01±0.01	98.06±0.00	98.08±0.01	98.08±0.01
	APAN	97.08±0.37	96.79±0.42	96.99±0.08	97.52±0.35	97.48±0.33

Table 6: Per-seed test AP (%) on the benchmark graphs. LastFM, Wiki-Talk, and Stack-Overflow: paired runs of the full-precision pipeline and QTem from the same initialization, sampler, and negatives; Δ: mean paired difference with the p-value of a paired t-test. GDELT: bit-plane packing is lossless, so QTem reads exactly the full-precision inputs (*: seed 7).

		Seed 0		Seed 1		Seed 2*		Mean			
Graph	Model	fp32	QTem	fp32	QTem	fp32	QTem	fp32	QTem	Δ	p
LastFM	TGN	82.78	83.79	84.10	83.69	84.69	84.72	83.86	84.07	+0.21	0.66
	TGAT	69.72	69.62	69.89	70.09	68.94	69.13	69.52	69.61	+0.10	0.44
	APAN	69.44	69.02	69.21	69.40	69.69	69.18	69.45	69.20	−0.25	0.38
Wiki-Talk	TGN	86.25	90.71	91.72	90.15	92.13	88.96	90.03	89.94	−0.10	0.97
	TGAT	86.94	86.87	86.81	86.80	86.80	86.93	86.85	86.87	+0.02	0.81
	APAN	86.21	87.06	86.96	85.80	86.70	86.45	86.62	86.43	−0.19	0.77
Stack-Overflow	TGN	85.93	86.85	85.89	85.53	85.97	85.10	85.93	85.83	−0.10	0.87
	TGAT	87.44	87.66	87.35	87.62	87.52	87.72	87.44	87.66	+0.23	0.01
	APAN	69.22	64.76	78.65	62.73	67.95	66.83	71.94	64.78	−7.16	0.25
GDELT	TGN	98.17		97.11		96.40		97.23		lossless	
	TGAT	98.09		98.07		98.08		98.08		lossless	
	APAN	97.15		97.56		97.84		97.52		lossless	

Table 7: Paired runs on real features (test AP, %; mean ± standard deviation over three seeds). Δ: mean paired difference to full precision, with the p-value of a paired t-test; 4-bit: four-bit codes with one scale per row.

			QTem, 8-bit		4-bit	no features	
Graph	Model	fp32	Δ	p	Δ	Δ	p
Wikipedia	TGN	98.42±0.19	+0.08	0.51	−0.03	+0.03	0.75
	TGAT	89.87±0.01	+0.04	0.71	+0.11	−0.15	0.32
	APAN	94.45±0.36	+0.03	0.93	+0.23	−0.66	0.04
Reddit	TGN	99.57±0.05	+0.05	0.37	+0.01	−0.97	0.007
	TGAT	96.52±0.02	+0.07	0.61	+0.09	−4.53	0.003
	APAN	96.88±1.45	+2.07	0.14	+0.50	−1.36	0.57

by automatic differentiation the Jacobian of the memory update with respect to every memory block that it reads and to the event features. TGN's GRU reads the node's own memory, as hidden state and as the first part of its message (**A** sums both Jacobians), the other endpoint's memory ($\mathbf{B}_1$), and the event feature (**C**). APAN's transformer updater reads the node's memory (**A**) and the two endpoint memories and the event feature of each of its ten mailbox entries (20 blocks $\mathbf{B}_j$, ten blocks of **C**). Our re-implementations reproduce the modules' outputs up to 9.5×10^{-7}. Table 11 summarizes the constants over all sampled updates of nodes whose memory is no longer empty.

Both updaters contract in mean square: g is at most 0.84 for TGN, and for APAN it exceeds one in a single sampled update. The gains do not drift over the stream; their medians are 0.80, 0.80, and 0.80 for TGN and 0.29, 0.28, and 0.28 for APAN in the training, validation, and test segments. With the root-mean-square values of the event features (1.00 for both models) and of the memories (0.74 for TGN, 0.78 for APAN), the steady state $l\,\sigma_x/\sqrt{1-g^2}$ predicts the ratios in the last rows from the median constants, in agreement with the replay within a factor of 1.3 for TGN and within 2% for APAN.

G TIMING DETAILS

End-to-end comparison. For the epoch times in Table 2, every run trains with the default configuration on one GPU, without validation, and records the wall-clock time of every training epoch, from its start until the prefetching thread has been joined, including temporal sampling; runs are executed one at a time per GPU. QTem

Table 8: Mean reciprocal rank (%) with 49 random negatives per positive on the test split (mean ± standard deviation over three seeds), and the mean paired difference with its 95% confidence interval and paired t-test.

Graph	Model	fp32	QTem	Δ [95% CI]	p
LastFM	TGN	37.83±1.28	38.32±0.59	+0.49 [−2.46, +3.43]	0.55
	TGAT	20.29±0.28	20.42±0.15	+0.13 [−0.28, +0.54]	0.30
	APAN	17.34±0.20	17.01±0.30	−0.33 [−0.99, +0.33]	0.16
Wiki-Talk	TGN	55.90±11.12	57.91±3.32	+2.01 [−33.8, +37.9]	0.83
	TGAT	52.27±0.09	52.35±0.31	+0.08 [−0.60, +0.77]	0.66
	APAN	46.39±0.76	46.16±1.86	−0.22 [−3.25, +2.80]	0.78
Stack-Overflow	TGN	50.03±0.13	48.79±1.84	−1.24 [−5.71, +3.23]	0.35
	TGAT	54.35±0.17	55.05±0.26	+0.70 [+0.43, +0.97]	0.01
	APAN	23.47±6.55	16.68±0.26	−6.79 [−23.4, +9.86]	0.22
Wikipedia	TGN	79.63±2.44	79.50±1.50	−0.13 [−4.55, +4.29]	0.91
	TGAT	59.38±6.87	55.72±6.04	−3.66 [−35.7, +28.4]	0.67
	APAN	58.08±1.56	58.47±4.44	+0.39 [−6.76, +7.54]	0.84
Reddit	TGN	91.27±0.72	91.85±0.78	+0.58 [−2.02, +3.18]	0.44
	TGAT	74.25±0.26	74.33±0.77	+0.08 [−2.46, +2.62]	0.91
	APAN	68.84±10.61	82.40±2.47	+13.6 [−9.95, +37.1]	0.13
All 45 pairs				+0.38 [−1.55, +2.30]	0.70

Table 9: Lock-step replay on LastFM (mean over three trained models; deviations relative, in %).

		Input	Logit	Flips	Memory		|ΔAP|
Model	Variant	err.	dev.		median	max	(pts)
TGN	QTem, $b = d$	0.65	0.21	0.10	0.09	0.57	≤0.002
	QTem, $b = 4$	0.31	0.10	0.05	0.04	0.27	≤0.001
	4-bit, $b = d$	11.7	3.79	1.80	1.69	10.2	≤0.064
	FP32 vs. FP32	0	3.49	1.36	6.03	59.0	≤0.040
APAN	QTem, $b = d$	0.65	0.02	0.00	0.04	1.02	≤0.003
	QTem, $b = 4$	0.31	0.02	0.00	0.02	4.72	≤0.001
	4-bit, $b = d$	11.7	0.41	0.01	0.81	8.68	≤0.018
	FP32 vs. FP32	0	0.42	0.01	1.14	18.4	≤0.009

(records in GPU memory, codecs of Table 4) runs directly before and directly after a full-precision run on the same GPU, with the same number of host threads, and its time is the mean over all epochs of both runs; its peak GPU memory (Section 6.2) is the memory allocated by PyTorch during training in these runs. TGL uses eight sampler threads, four intra-op threads (on LastFM, the faster of 4 and 32), and all data in host memory. SIMPLE uses the faster of the timed buffer sizes, all data on LastFM and Wiki-Talk and its default of 10% of the data on Stack-Overflow and GDELT, and its offline planning is not included. ETC's time is the fastest over several sessions and host-thread counts. On Stack-Overflow and GDELT, TGL, Orca, and Zebra train the first 1,000 mini-batches (TGAT: 500) of two epochs, and each epoch is scaled linearly to all of its mini-batches; repeating these runs over complete epochs leaves the fastest system of every configuration unchanged.

Feature store. The store in Table 2 is the size of the static node and edge tables that a system holds during training, in the format of its timed runs; it does not depend on the model. ETC holds 32-bit floats and converts GDELT's binary attributes to them while loading (139.3 GB), and SIMPLE, which loads features with ETC's code, holds the same tables. TGL holds 32-bit floats as well, except on GDELT, where its reader memory-maps tables with one byte per binary value (34.8 GB). Orca and Zebra hold 32-bit tables, except for GDELT's edge table, which they keep in host memory with one byte per value. QTem holds the records of Section 3, $d + 2\lceil d/b \rceil$ bytes per row for 8-bit codes with binary16 block scales and $\lceil d/8 \rceil$ bytes per row for the bit planes of GDELT, in GPU memory, and keeps no copy in host memory. On Wiki-Talk, all systems hold 172-dimensional random edge features.

Traffic. Table 14 lists the host-to-GPU payload of the first training epoch as counted in Section 2.3 (static features and dynamic state), for QTem from its timing runs and for the full-precision path (FP32) from runs with the same model, batch size, data, and seed. Where both are counted, their dynamic-state bytes agree within 0.003%.

Orca and Zebra on TGAT and APAN. Both systems implement TGN only. Table 13 gives the epoch times of ports of both to TGAT and APAN, timed by the same rules: QTem is faster than both ports in 15 of the 16 comparisons. TGAT is the memory-less case of their architecture: the port removes the node memory, uses the node features as layer-0 representations, and keeps their two attention layers along with Orca's reuse of embeddings and Zebra's PageRank-based neighbors. APAN keeps ten mails per node, updates memories with TGL's attention-based updater, and delivers every mail to the node and to the neighbors that each system samples.

Data-path comparisons. Table 12 lists the epoch time of every timing run behind Section 6.5. In every round, all variants of a configuration trained one epoch back to back on the same GPU, in listed order in odd rounds and in reverse order in even rounds, so that a linear drift over time affects all variants alike. Between rounds, single epochs of a configuration vary by up to 30% for FP32 and 57% for uint8+pin, which gather rows on the host, but by at most 19% for QTem-G. Speedups are computed per round against FP32 in the same round and summarized by their median.

Stack-Overflow-TGN. TGN moves node memories and mailboxes besides static features, and the pipeline splits the work of a batch between two threads: the prefetching thread constructs the message-flow graphs and gathers, copies, and reconstructs the static rows, 305 s per epoch at full precision, while the main thread gathers and copies memories and mailboxes, computes, and updates memories, 610 s per epoch, and waits for the prefetching thread for only 6 s. The static path runs concurrently with the main thread, so the less static work the prefetching thread performs, the faster the main thread completes its own. Across the five variants, the static gathers, copies, and reconstruction take 162 s per epoch under FP32, 132 s under FP32+pin, 97 s under QTem-H+pin, 45 s under FP16-G, and 47 s under QTem-G, and the main thread's computation, memory updates, and reconstruction take 413 s, 375 s, 367 s, 340 s, and 341 s, respectively (medians over four rounds). QTem-G thus gains 1.17× over FP32, about as much as FP16-G (1.18×), with 18.3 GB of GPU memory instead of 24.0 GB.

Smaller graphs. The full-precision stores of LastFM and Wiki-Talk fit on the GPU, so their strongest baseline keeps full-precision tables on the GPU (FP32-G). On these graphs, QTem-G trains an epoch 1.05–1.20× faster than FP32 and as fast as FP32-G (0.98–1.01×; medians over four rounds) with a feature store 2.7–3.9× smaller.

Table 10: Lock-step replay on real features (mean over three trained models). Dev.: relative deviation of the link logits from the full-precision copy over the test segment (%); flips: share of link predictions whose sign changes (%); ΔAP: change of the test AP (points; for QTem the largest magnitude over the three models). FP32 vs. FP32 compares two copies of the unmodified models; for TGAT, which has no memory, they agree exactly.

		QTem, 8-bit			4-bit codes		shuffled rows		zeros		FP32 vs. FP32
Graph	Model	dev.	flips	|ΔAP|	dev.	ΔAP	dev.	ΔAP	dev.	ΔAP	dev.
Wikipedia	TGN	0.22	0.024	0.001	8.9	−0.06	16.2	−0.17	22.7	−0.43	5.35
	TGAT	0.43	0.052	0.005	13.3	−0.26	27.2	−0.83	41.1	−2.88	0
	APAN	0.19	0.011	0.006	4.4	−0.32	12.9	−0.54	10.8	−0.84	1.82
Reddit	TGN	0.15	0.009	<0.001	22.4	−1.75	20.1	−0.35	24.7	−0.37	2.97
	TGAT	0.48	0.112	<0.001	11.3	−0.25	80.7	−12.04	114.0	−23.13	0
	APAN	0.29	0.037	0.004	36.6	+0.90	78.4	−1.48	101.5	−2.49	1.64

Table 11: Jacobian constants of trained memory updaters on LastFM: worst-case gain $\rho = \|\mathbf{A}\|_2 + \sum_j \|\mathbf{B}_j\|_2$ (for TGN the smaller of this and $\sqrt{2}\,\|[\mathbf{A}\,\mathbf{B}_1]\|_2$), own-memory block $\|\mathbf{A}\|_2$, feature gain $L = \sum \|\mathbf{C}\|_2$, mean-square gains g and l (Section 4), and the ratio of relative memory deviation to relative input error predicted by the mean-square steady state and measured in the lock-step replay (Table 9). Median (99th percentile, maximum).

	TGN	APAN
Sampled updates	25,427	5,086
ρ	1.28 (1.86, 2.24)	3.07 (5.49, 20.3)
$\|\mathbf{A}\|_2$	1.03 (1.23, 1.40)	0.83 (1.72, 10.5)
L	0.24 (0.53, 0.66)	0.85 (1.31, 3.53)
g	0.80 (0.82, 0.84)	0.29 (0.45, 1.28)
Share with $g < 1$	100%	99.98%
l	0.049 (0.099, 0.23)	0.052 (0.086, 0.21)
Ratio, predicted	0.11	0.069
Ratio, measured	0.14	0.068

H COMPARISON WITH SIMPLE

Setup. SIMPLE [2] decides offline, from the access pattern of one sampled epoch, which rows of each table stay in a GPU buffer during which mini-batches, and updates the buffer between batches; all other rows are gathered from host memory at full precision. For our data-path comparisons, we ran it with the model configurations of Table 4 and the feature tables of our other runs. The buffer holds 10% of the data, SIMPLE's default, except on GDELT, where it receives the 4.4 GB that QTem-G occupies. We completed the APAN path of its training and evaluation loops as in TGL and corrected three errors of its released buffer management that corrupt buffered rows: buffer slots are assigned by positions in a masked array but checked against the length of the unmasked one, evicted node memories and mailboxes are not written back, and batches smaller than the buffer are assembled inside it. With the corrections, which do not change the placement plan, every buffered feature row matches its host copy, and SIMPLE trains TGAT exactly like TGL: every corrected SIMPLE run on TGAT that shared its seed with a TGL run reached the same test AP. Table 2 lists SIMPLE as released; the corrections only add work, so its times there favor SIMPLE.

Results. Table 15 breaks SIMPLE's epoch times in our data-path comparisons down by its own timers. On Stack-Overflow-TGAT, SIMPLE takes 634 s per epoch against 409 s for FP32 and 269 s for QTem-G in the same round. Its buffer reduces the rows gathered from host memory to 64 s of gathering per epoch, but managing the buffer, that is, deriving the buffer contents of each batch from the plan, maintaining flags and indicators, and updating the buffer, costs 183 s, more than the entire feature path of FP32. On GDELT-TGAT, with a buffer as large as the QTem records, SIMPLE takes 712 s against 621 s for FP32: the buffer leaves 159 s of gathering per epoch, and managing it adds another 112 s. On Stack-Overflow-TGN, where the buffer also holds node memories and mailboxes, SIMPLE takes 1115 s against 615 s for FP32, and buffer management costs 386 s per epoch. In each of the three large configurations, the single SIMPLE epoch took longer than every one of the four FP32 epochs (Table 12).

I MICROBENCHMARK DETAILS

The microbenchmark of Figure 11, summarized in Section 6.5, fetches R distinct, sorted row IDs drawn uniformly from tables of 4–24 million rows, i.e., far larger than the CPU caches, and synchronizes the GPU after every stage. Each variant is repeated 25 times for $R < 2^{18}$ and 10 times otherwise after three warm-up runs, and we report medians. Table 16 lists the latency at $R = 2^{18}$, including two variants that the figure omits. With pinned staging buffers, full-precision fetches become 2.7–3.5× faster, and QTem keeps its lead at equal copy mode: 2.2–3.7× for dense tables and 4.7–6.3× for binary tables. Storing codes and scales in two separate arrays and gathering them with the pipeline's advanced indexing, as a direct port of a row-wise 8-bit quantizer would do, is 1.1–1.5× slower than the fixed-width records, almost entirely because of the host-side gather. GPU-resident records remove the host gather altogether: at $R = 2^{14}$, close to the per-batch row counts of Stack-Overflow, a fetch from the dense $d = 172$ table with four-entry blocks takes 5.3 ms at full precision, 0.7 ms with a pinned staging buffer, and 0.15 ms from GPU-resident records.

J SCALE RANGE AND BINARY32 SCALES

Theorem 4.1 requires normal binary16 scales, which every block of magnitude between $Q \cdot 2^{-14}$ and $Q \cdot 65504$ receives. We tested the codec in isolation on adversarial inputs: all-zero blocks, blocks with magnitudes of 10^{-40}, 10^{-6}, 1, 10^{7}, and close to the binary32 maximum, and non-uniform blocks. All-zero blocks decode exactly,

Table 12: Every timing run of Section 6.5. In every round, all variants of a configuration trained one epoch back to back on the same GPU, in listed order in odd rounds and in reverse order in even rounds. Epoch time of every round, its median (Med.), the median per-round speedup over FP32 (Sp.), and the peak GPU memory allocated during the epoch (GB; SIMPLE: its own report). Each configuration lists the variants that apply to its tables: one byte per binary value on GDELT, half precision on Stack-Overflow, and full-precision tables on the GPU on LastFM and Wiki-Talk, whose stores fit there.

	Epoch time per round (s)						
Variant	1	2	3	4	Med.	Sp.	GPU
GDELT, TGAT							
FP32	621	579	679	643	632	1.00	1.1
FP32+pin	545	575	559	541	552	1.16	1.1
uint8+pin	799	510	526	525	525	1.18	1.1
uint8-G	509	512	521	455	510	1.26	35.9
SIMPLE	712	*first round only*			712	0.87	8.1
QTem-H+pin	560	518	556	466	537	1.17	1.1
QTem-G	491	488	523	461	490	1.28	5.5
Stack-Overflow, TGAT							
FP32	409	354	349	315	351	1.00	0.6
FP32+pin	384	366	362	296	364	1.02	0.6
FP16+pin	349	277	322	268	299	1.17	0.6
FP16-G	261	251	270	259	260	1.35	23.3
SIMPLE	634	*first round only*			634	0.64	9.7
QTem-H+pin	279	435	294	275	287	1.17	0.6
QTem-G	269	312	286	275	281	1.18	17.6
Stack-Overflow, TGN							
FP32	615	739	656	651	653	1.00	1.3
FP32+pin	634	655	635	573	634	1.08	1.3
FP16-G	620	604	570	534	587	1.18	24.0
SIMPLE	1115	*first round only*			1115	0.55	11.4
QTem-H+pin	606	625	556	594	600	1.14	1.3
QTem-G	618	611	522	573	592	1.17	18.3

	Epoch time per round (s)						
Variant	1	2	3	4	Med.	Sp.	GPU
LastFM, TGN							
TGL	36.0	38.3	35.3	37.0	36.5	0.58	0.4
FP32	19.3	21.8	22.8	22.3	22.0	1.00	0.5
FP32+pin	17.1	20.5	21.1	23.9	20.8	1.07	0.5
FP32-G	16.9	18.9	18.0	19.2	18.4	1.16	1.2
SIMPLE	18.6	19.6	18.8	18.0	18.7	1.16	0.4
QTem-H+pin	17.8	21.9	21.1	21.2	21.2	1.07	0.5
QTem-G	15.8	19.6	18.9	18.6	18.8	1.20	0.7
LastFM, TGAT							
TGL	13.7	9.9	10.2	10.9	10.5	0.47	0.6
FP32	6.6	4.6	6.1	4.7	5.4	1.00	0.6
FP32+pin	7.4	5.5	7.4	4.6	6.4	0.86	0.6
FP32-G	5.7	4.0	5.4	4.3	4.9	1.14	1.3
SIMPLE	6.9	4.9	5.5	6.4	6.0	0.95	0.7
QTem-H+pin	6.7	4.3	6.3	4.7	5.5	0.98	0.6
QTem-G	6.0	4.5	5.1	4.1	4.8	1.12	0.8
Wiki-Talk, TGN							
FP32	62.8	70.8	60.8	63.0	62.9	1.00	0.5
FP32+pin	63.4	66.1	60.4	62.4	62.9	1.01	0.5
FP32-G	57.5	59.7	59.3	59.6	59.5	1.07	1.3
SIMPLE	80.1	81.0	88.8	90.7	84.9	0.74	0.7
QTem-H+pin	59.9	63.8	63.1	63.2	63.1	1.02	0.5
QTem-G	57.2	58.5	58.1	64.6	58.3	1.07	0.8
Wiki-Talk, TGAT							
FP32	36.1	34.3	35.4	34.2	34.8	1.00	0.2
FP32+pin	35.7	34.6	33.5	37.5	35.1	1.00	0.2
FP32-G	33.2	32.0	32.4	32.5	32.5	1.08	1.0
SIMPLE	49.8	50.8	51.1	48.2	50.3	0.70	0.4
QTem-H+pin	38.2	34.1	33.4	36.9	35.5	0.98	0.2
QTem-G	33.8	33.6	32.8	33.4	33.5	1.05	0.5

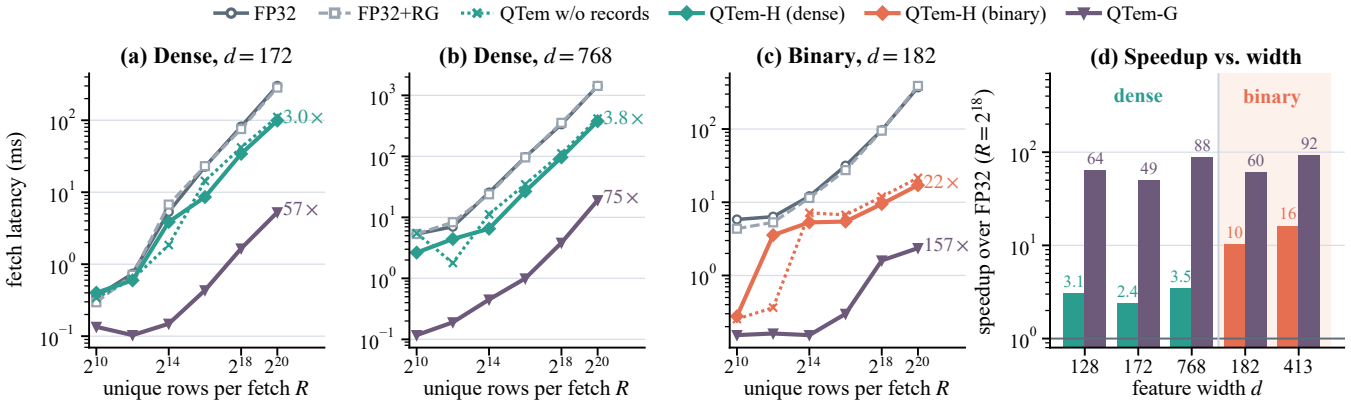


Figure 11: Latency of fetching R deduplicated rows into GPU memory as decoded rows (median of repeated runs). (a)–(c) Latency against R; labels give the speedup over FP32 at $R = 2^{20}$. (d) Speedup over FP32 at $R = 2^{18}$ per table width.

and blocks too small for a normal binary16 scale decode with an absolute error of at most their magnitude. Binary32 scales extend the bound to every block whose scale is a normal binary32 number, that is, of magnitude at least $Q \cdot 2^{-126} \approx 1.5 \times 10^{-36}$ for 8-bit codes: the product of a code and a scale then adds one rounding, and the argument of Theorem 4.1 yields $|x_{ik} - \hat{x}_{ik}| \le \hat{s}_{ij}(\frac{1}{2} + (2Q + \frac{1}{2})u)$. The largest error we measured over all regimes with normal binary32 scales is $0.49999\,\hat{s}_{ij}$. Binary32 scales cost two more bytes per block: with one scale per row, the reduction for $d = 128$, 172, and 768 changes from 3.94, 3.95, and 3.99× to 3.88, 3.91, and 3.98×. Binary16

Table 13: Epoch time (s) of ports of Orca and Zebra to TGAT and APAN, timed like Table 2, and of QTem (Table 2). *Extrapolated from the first 1,000 mini-batches (TGAT: 500) of two epochs.

	LastFM		Wiki-Talk		Stack-Ov.		GDELT	
	TGAT	APAN	TGAT	APAN	TGAT	APAN	TGAT	APAN
Orca	9.91	14.5	72.1	228.2	1391*	2007*	5095*	1245
Zebra	36.6	41.6	212.4	418.5	2942*	3209*	9838*	4900*
QTem	3.28	13.1	17.7	121.4	175.6	1053	542.5	1699

Table 14: Host-to-GPU traffic in the first training epoch (GB) with the default configurations and seed 0, without the one-time copy of the records of QTem while loading. Dynamic state: memories, mailboxes, and their timestamps.

		Static features		Dynamic	Total	
Graph	Model	FP32	QTem	state	FP32	QTem
LastFM	TGN	4.29	0	2.00	6.29	2.00
	TGAT	1.16	0	0	1.16	0
	APAN	0.46	0	13.1	13.6	13.1
Wiki-Talk	TGN	29.1	0	28.4	57.5	28.4
	TGAT	18.7	0	0	18.7	0
	APAN	3.77	0	140.7	144.5	140.7
Stack-Overflow	TGN	396.4	0	408.8	805.2	408.8
	APAN	30.6	0	1211	1242	1211
GDELT	TGN	1067	0	378.4	1445	378.4
	TGAT	808.8	0	0	808.8	0
	APAN	93.6	0	2311	2405	2311

Table 15: Where SIMPLE's epoch time goes (seconds, SIMPLE's own timers; medians over the rounds in which SIMPLE ran: the first round on GDELT and Stack-Overflow, all four rounds on the smaller graphs): sampling, construction of message-flow graphs, gathering rows that are not buffered, and buffer management (deriving the buffer contents of each batch from the plan, flags, indicator and buffer updates, and for TGN writing back memories and mailboxes), with SIMPLE's peak GPU memory (GB). FP32: epoch time over the same rounds and its feature path (host gather plus copy, median over all rounds; *: for TGN, the copy step also gathers and copies memories and mailboxes).

		SIMPLE						FP32	
Graph	Model	Epoch	Sample	MFG	Load	Buffer	GPU	Epoch	Feat.
GDELT	TGAT	712	121	44	159	112	8.1	621	239
Stack-Overflow	TGAT	634	58	32	64	183	9.7	409	157
Stack-Overflow	TGN	1115	60	34	180	386	11.4	615	240*
Wiki-Talk	TGAT	50	5.5	3.8	4.3	8.5	0.4	35	5.4
Wiki-Talk	TGN	85	5.5	4.0	12	17	0.7	63	15*
LastFM	TGAT	6.0	1.2	0.5	1.4	0.5	0.7	5.4	1.4
LastFM	TGN	19	1.4	1.2	1.6	3.4	0.4	22	7.8*

scales suffice for the tables we use: no scale of the two LIWC tables and of one million rows of benchmark-style features leaves the binary16 normal range, and only 4 of 192 million four-entry blocks of the MAG sample underflow.

Table 16: Fetch latency (ms) at $R = 2^{18}$, median of repeated runs. "pin": pinned staging buffer; "sep.": codes and scales in separate arrays gathered with advanced indexing (binary tables: packed rows gathered with advanced indexing).

	full precision			QTem			
Table	FP32	+RG	+pin	sep.	QTem-H	+pin	QTem-G
dense, $d = 128$	58.7	63.8	20.8	27.9	19.1	9.4	0.91
dense, $d = 172$	81.8	75.5	29.9	32.0	22.5	8.1	1.01
dense, $d = 172$, $b = 4$	81.8	75.5	29.9	42.0	34.2	11.9	1.65
dense, $d = 768$	335.4	354.4	101.7	111.9	96.3	37.1	3.83
binary, $d = 182$	96.4	95.0	32.2	11.9	9.5	6.8	1.60
binary, $d = 413$	209.9	209.2	60.7	14.3	13.0	9.6	2.28

REFERENCES

[1] Shihong Gao, Yiming Li, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. ETC: Efficient Training of Temporal Graph Neural Networks over Large-Scale Dynamic Graphs. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1060–1072. https://doi.org/10.14778/3641204.3641215

[2] Shihong Gao, Yiming Li, Xin Zhang, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. SIMPLE: Efficient Temporal Graph Neural Network Training at Scale with Dynamic Data Placement. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–25. https://doi.org/10.1145/3654977

[3] Hongkuan Zhou, Da Zheng, Israt Nisa, Vasileios Ioannidis, Xiang Song, and George Karypis. 2022. TGL: A General Framework for Temporal GNN Training on Billion-Scale Graphs. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1572–1580. https://doi.org/10.14778/3529337.3529342